

Iteration 3 Worksheet

What technical debt has been cleaned up ?

We [fixed an Open/Closed Principle \(OCP\) violation](#) from our last Iteration. This was an instance of prudent, inadvertent technical debt because we initially didn't realize we were introducing technical debt. However, upon review, we identified the issue, understood where we went wrong, and now have a better approach for implementing it correctly in the future.

What technical debt did you leave? (florence)

We decided to leave the construction of the Item object with multiple parameters instead of restructuring it using an Abstract Factory pattern due to limited time constraints. This change was deferred to Iteration Four as a case of prudent, deliberate technical debt.

Discuss a Feature or User Story that was cut/re-prioritized

When did you change the priority of a Feature or User Story? Why was it re-prioritized? Provide a link to the Feature or User Story. This can be from any iteration.

If you didn't cut anything, find a good software engineering meme and provide that.

One user story we ended up removing was the ability to **sort the listings** by price, or date posted. This was originally planned for **Iteration 2**, but we decided to cut it because it was making the UI feel too cluttered and messy. We already had category filters and bookmark options, and adding a sort icon on top of that just made the page harder to use and navigate.

We realized it's better to keep things clean and simple for the user, rather than stuffing in every possible feature. It was a group decision, and we all agreed it improved the overall experience. The user story was closed and marked as technical debt in case we want to revisit it later.

References:

- [Sort - User Story](#)
- [Release Notes](#)
- [Group discussion Meeting notes](#)

System test

We wrote a system test called "ProfileManagementTest". It is designed to verify that a user can log in and successfully change their first name in the profile section of the application. The test follows a structured approach to ensure it is reliable, repeatable, and not flaky.

To prevent flakiness, we implemented the following:

UI Synchronization: Espresso's `matches()` method is used to verify elements before interacting with them, ensuring the UI has loaded.

Soft Keyboard Handling: The test explicitly closes the keyboard using `closeSoftKeyboard()` to prevent UI obstruction.

Consistent Test Data: A fixed test account (`parkerp@myumanitoba.ca`) ensures that changes are predictable and reproducible.

Clear and Update Strategy: The test first clears the first name field before entering a new value to avoid input inconsistencies.

Verification at Every Step: Assertions are placed after key actions to confirm UI changes before proceeding.

The System test is linked here: [ProfileManagementTest.java](#)

System test, untestable

Device-Specific Issues: Testing on different emulator devices introduced unexpected problems. For example, when testing on a phone, We had to use the `scrollTo()` action to reach a button at the end of a page. However, when switching to a tablet, We couldn't access a button even after scrolling due to a navigation bar obstructing the view.

Timing and Action Delays: Some actions needed to be delayed to ensure that the previous step had fully completed before moving forward. Failing to manage this timing correctly sometimes resulted in crashes or flaky tests.

Soft Keyboard Handling: Remembering to close the keyboard after typing was a recurring challenge. If not handled properly, the keyboard would obstruct elements, and the fast execution speed of tests would make interactions unreliable.

Android Studio Performance: The speed at which actions are performed in system tests sometimes made debugging difficult, as failures happened quickly and were hard to diagnose.

SOLID

[SearchActivity.java](#)

[Issue #88](#)

The `SearchActivity` class in its current form violates the Single Responsibility Principle (SRP) by handling multiple concerns within a single class. While it is appropriate for an activity to manage UI interactions, this class also performs business logic by directly retrieving data through `AccessProducts`, handles input validation by checking if the search field is empty, and manages navigation to other activities using explicit `Intent` construction. Each of these responsibilities like data access, validation, and navigation could independently change for different reasons which can lead to a fragile and less maintainable design. To adhere to SRP, the business logic should not be imported to the `presentation` package, it's a code SMELL, the validation logic can be extracted into a utility class like `InputValidator`, and navigation should be centralized using a `NavigationHelper` or `IntentFactory`.

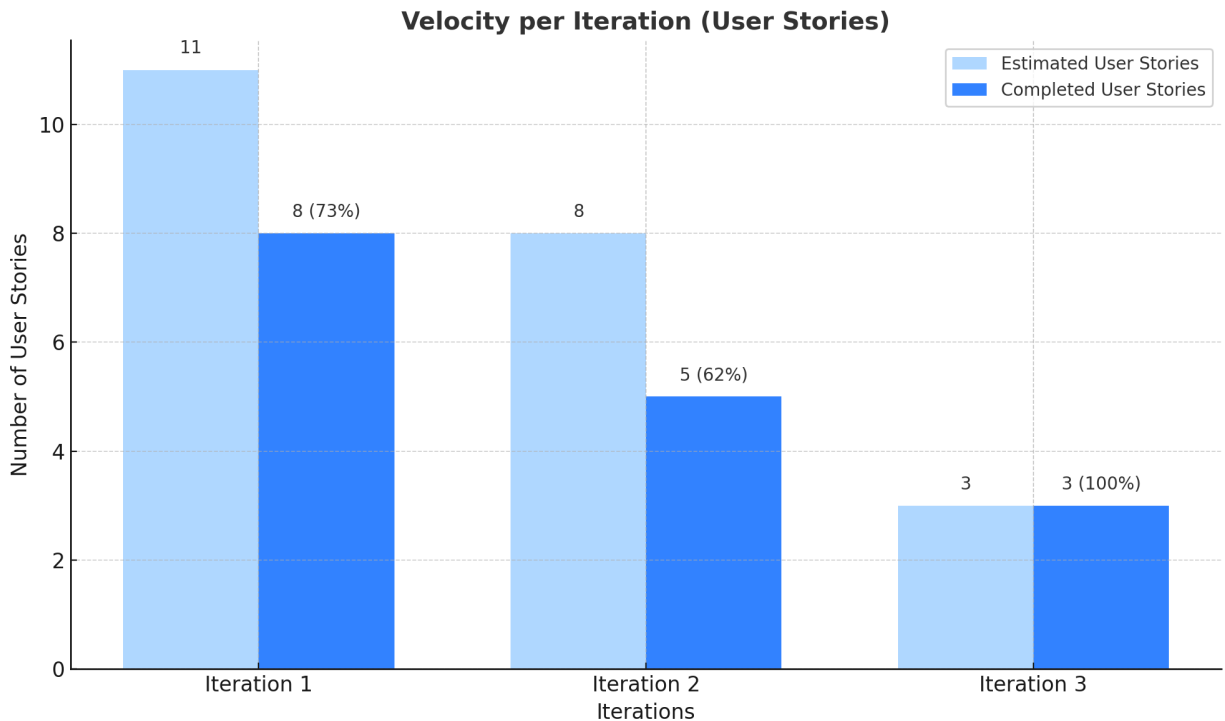
Velocity/teamwork

The team's project velocity over the three iterations reflects a shifting balance between feature development and technical debt management. In **Iteration 1**, the team planned 11 user stories and successfully completed 8, achieving approximately 73% of the intended workload. This iteration set a strong foundation with key features delivered on time.

In **Iteration 2**, the team planned 8 stories but completed only 5 (63%). This marked the beginning of growing technical challenges, as unresolved issues began to affect delivery. Despite this, important features such as bookmarking, categorization, and profile management were implemented.

By **Iteration 3**, only 3 stories were scoped, and while all were completed (100%), the sprint was largely focused on resolving accumulated technical debt. As a result, the app did not reach a fully stable or testable state by the end of the iteration, and new feature development had to be limited.

The velocity chart clearly illustrates this decline in throughput across the iterations, not just in completion percentage but also in the number of planned features. It highlights the reality of project dynamics — where maintaining code quality and stability can often take priority over speed of delivery in later stages.



Learned

What is one unexpected thing you learned by doing this project?

One unexpected thing we learned from this project was how important it is to manage time well and not just rush to finish things. At first, we were focused on just getting the work done, but we realized that writing clean, organized code actually saved us time later. Clean code is easier to understand, fix, and build on, while messy code just causes more problems.

Working in a group really helped too. Everyone brought different strengths, and we supported each other when things got tough. We also helped keep each other accountable for writing better code and avoiding bad practices. In the end, taking the time to do things properly, not just quickly, made the whole project smoother and more enjoyable.